No.

Date

Nama : Alya Haniyah

NIM : 24/543330/TK/60370

PROBLEM I

Snippet A

Line	cost / execution	number of execution	total cost
2.) sum=0	1	1	1
3.) int i=0	1	1	1
3.) i<n	1	n+1	n+1
3.) i++	1	n	n
4.) int j=0	1	n	n
4.) j<i	1	n(n-1)/2+n	n(n-1)/2+n
4.) j++	1	n(n-1)/2	n(n-1)/2
5.) arr[i]	1	n(n-1)/2	n(n-1)/2
5.) arr[j]	1	n(n-1)/2	n(n-1)/2
5.) + add	1	n(n-1)/2	n(n-1)/2
5.) +	1	n(n-1)/2	n(n-1)/2
5.) =	1	n(n-1)/2	n(n-1)/2

$$T(n) = 2 + (n+1) + 2n + \frac{n(n-1)}{2} + n + 6\left(\frac{n(n-1)}{2}\right)$$

$$T(n) = 4n^2 + n + 2$$

Big-O → $O(n^2)$

Best / worst case → Best = worst, tidak tergantung isi array

Dipindai dengan CamScanner

No.

Date

Fold

Snippet B Let $k = \lfloor \log_2 n \rfloor + 1$

Line	Cost / execution	number of execution	total cost
2.) lo = 0	1	1	1
2.) hi = n - 1	2	1	2
2.) target = 42	1	1	1
3.) found = false	1	1	1
4.) lo <= hi	1	k + 1	k+1
5.) hi - lo	1	k	k
5.) /2	1	k	k
5.) lo +	1	k	k
5.) ass mid	1	k	k
6.) arr [mid]	1	k	k
6.) compare ==	1	k	k
7.) arr [mid]	1	k	k
9.) compare <	1	k	k
8.) hi = mid - 1	2	k	2k

$T(n) = 1 + 2 + 1 + 1 + k + 1 + k + k + k + k + k + k + k + k + 2k$

$T(n) = 6 + 12k$

$T(n) = 12(\log_2 n + 1) + 6$ ▨

Big-O → $O(\log n)$ ▨

Best / worst case :

Best → target found. immediatly. ($k = 1$)

worst → search continuous untill space size 1 ($O \log n$) ▨

Big BOSS 6 mm

Dipindai dengan CamScanner

No.

Date

Snippet C

Line	cost	numb of exect	TOTAL cost
2.) count = 0	1	1	1
3.) i = 0	1	1	1
3.) i < n	1	1	1
3.) i++	2	n	2n
4.) j = i+1	2	n	2n
4.) j < n	1	$\frac{n(n-1)}{2} + n$	$n^2+n/2$
4.) j++	2	$\frac{n(n-1)}{2}$	$n^2-n/2$
5.) k = 0	1	$\frac{n(n-1)}{2}$	$n^2-n/2$
5.) k < n	1	$(\frac{n}{2}+1)(\frac{n(n-1)}{2})$	$n^3+n^2-2n/4$
5.) k += 2	2	$\frac{n}{2}(\frac{n(n-1)}{2})$	$n^3-n^2/2$
6.) if check	4	$\frac{n}{2}(\frac{n(n-1)}{2})$	$3n^3-3n^2/2$
7.) count ++	2	$\frac{n}{2}(\frac{n(n-1)}{2})$	$n^3-n^2/2$

$$T(n) = 1+1+1+2n+2n+\frac{n^2+n}{2}+\frac{n^2-n}{2}+\frac{n^2-n}{2}+\frac{n^3+n^2-2n}{4}+\frac{n^3-n^2}{2}+\frac{3n^3-3n^2}{2}+\frac{n^3-n^2}{2}$$

$$T(n) = \frac{11}{4}n^3 - \frac{3}{4}n^2 + 3n + 3$$

Big-O → $O(n^3)$

Best/worst case:

best → condition always false (increment never executed)

Worst → condition always true (increment always executed)

Big BOSS 6 mm

Dipindai dengan CamScanner

No.

Date

PROBLEM 3

(B.) Complexity Comparison Table

operation	Best Case	Worst Case	Average Case
Linear Search (unsorted)	O(1), The target is found at the first element so only one comparison is needed.	O(n), The algorithm must check every element if the target is last or not present.	O(n), on average about half of the array elements must be checked sequentially.
Linear Search (sorted, early exit)	O(1), The target is found immediately at the first position.	O(n), The search may still need to scan all elements if the target is last or larger than all elements.	O(n), even with sorting, the algorithm may still check many elements sequentially before finding the target.
Binary Search (sorted)	O(1), The target happens to be at the middle element on the first comparison	O(logn), Each comparison halves the remaining search space until the element is found or not found.	O(logn), On average the search repeatedly halves the array, requiring logarithmic steps.

-> Linear Search worst case $\approx n$

Binary search worst case $\approx \log_2 n$

-> Binary search lebih efisien jika $\log_2 n < n$

-> Binary search becomes more efficient starting at $n = 2$ ▨

Big BOSS 6 mm

Dipindai dengan CamScanner

No.

Date

PROBLEM 4

A. Trace

- Starting from capacity = 2
- If size = capacity, then capacity = 2x capacity
- cost : Normal append = 1

  Resize = 1 + old size

call ※	Value Pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	1+2=3
4	40	4	4	1
5	50	5	8	1+4=5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	1+8=9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

B. Total cost

- Sum all value:

  $T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1 = 26$

- Average cost :

  $\frac{T_{12}}{12} = \frac{26}{12} \approx 2.167$

- Resize menyalin : $2+4+8+\dots+2^k$ (geometric series). Jumlahnya $2+4+8+\dots+2^k = 2^{k+1}-2$, Karena $2^k \approx n$, maka $T_{copy} < 2n$. Jadi,

Total cost : $T(n) = n + T_{copy}$ Amortised cost : $\frac{T(n)}{n} = O(1)$

$T(n) = O(n)$.

Dipindai dengan CamScanner

No.

Date . .

Fold

C. Growth factor Comparison

-> new rules : capacity = old capacity + 4

-> capacity becomes 2 6 10 14 18 ...

1.) Trace

Call #	Value	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	1+2 = 3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	1+6 = 7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	1+10 = 11
12	120	12	14	1

New T_{12} = 1+1+3+1+1+1+7+1+1+1+11+1 = 30

2.) Comparison

doubling = 26 } n kecil perbandingannya kecil.

+4 fixed = 30 } n besar perbandingan sgt signifikan.

3.) General Analysis

-> doubling strategy → copywork : 2 + 4 + 8 + 16 + ...

Deret Geometri : < 2n

$T(n) = O(n)$

Amortized : $O(1)$.

-> fixed increment strategy → Resized terjadi di 2, 6, 10, 14, ...

copywork : 2 + 6 + 10 + 14 + ... (Deret aritmatika)

BIG BOSS 6 mm

Dipindai dengan CamScanner

No

Date

- Jika Jumlah resize $\approx n/4$, then : $\sum_{i=0}^{n/4} (2+4)$.

- Rumus deret aritmatika : $m/2$ (first + last)

  karena $m \propto n$ dan last term $\propto n$, maka

  $T(n) = O(n^2)$

- amortized cost : $\frac{O(n^2)}{n} = O(n)$

final conclusion

Strategy	Resize growth	Series type	Total cost	Amortized cost
Doubling	$\times 2$	Geometric	$O(n)$	$O(1)$
+4 Increment	+4	Arithmetic	$O(n^2)$	$O(n)$

The doubling strategy results in a geometric series of copy costs, giving a total cost of $O(n)$ and an amortized cost of $O(1)$ per pushBack.
in contrast, the fixed-increment strategy produces an arithmetic series of copy costs, resulting in $O(n^2)$ total cost and $O(n)$ amortized cost.
Therefore, doubling capacity is significantly more efficient.

BigBOSS 6 mm

Dipindai dengan CamScanner